

Introduction

This practice consisted of turning a Raspberry Pi 4 into a self-contained wireless access point that hosts a small web server, so that any phone or laptop joining its network can drive, from a browser, the GPIO circuit built in the previous practice: seven LEDs driven through a ULN2003 Darlington array and a seven-segment display driven through a 74LS47 BCD decoder. The objective was to integrate three layers that had so far been handled separately (network configuration at the operating-system level, an HTTP service written in Python, and the hardware control logic) into a single embedded system that reacts to requests arriving over the air instead of commands typed on a local terminal.

Background

As Kurose and Ross [1] explain, in an infrastructure-mode wireless LAN every station associates with an access point, which identifies its network through a service set identifier (SSID) advertised periodically in beacon frames. Gast [2] adds that the 802.11i amendment, commercialized as WPA2, derives the encryption keys of each session from a pre-shared passphrase of 8 to 63 characters, which is why a hotspot can be secured with nothing more than an SSID and a password.

A station that joins the network still needs an IP address. According to Kurose and Ross [1], the Dynamic Host Configuration Protocol (DHCP) assigns it through a four-message exchange (discover, offer, request and acknowledgment), and every address is granted as a lease of finite duration. Tanenbaum and Wetherall [3] point out that the block 192.168.0.0/16 is reserved for private networks and is never routed on the public Internet, which makes it suitable for an isolated network such as the one built here.

Once connected, the client talks to the Pi through HTTP. Kurose and Ross [1] describe it as a stateless request–response protocol running over TCP, whose servers listen on port 80 by default; a GET request retrieves the object named in its URL, whereas a POST request carries the user’s input inside the entity body of the message. Monk [4] shows that such a web interface is a practical way to operate the GPIO outputs of a Raspberry Pi from any device on the same network.

Finally, Silberschatz *et al.* [5] note that a thread shares code, data and open files with the other threads of its process, and that multithreading keeps an application responsive, since the program can continue serving the user while one of its parts is busy in a lengthy operation. This property is essential here, because an LED animation is an endless loop while the server must keep accepting new commands.

Development

Access point. The package `python3-magic` was installed and, since the Pi runs Raspberry Pi OS based on Debian 13 (Trixie), the access point was defined with a single NetworkManager connection in *shared* mode, in which NetworkManager itself assigns addresses to the clients through DHCP and uses the static address of `wlan0` as the gateway:

```
nmcli connection add con-name AccessPoint type wifi ifname wlan0 \
  ssid Raspberry_Ortega 802-11-wireless.mode ap 802-11-wireless.band bg \
  wifi-sec.pairwise ccmp wifi-sec.proto rsn wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk 12345678 ipv6.method disabled ipv4.method shared \
  ipv4.address 192.168.1.254/24 ipv4.gateway 192.168.1.254 autoconnect yes
```

An earlier configuration based on a standalone dnsmasq service was discarded, and that service was disabled because it competes with NetworkManager for the DHCP port on wlan0. Remote access was kept through eth0 over SSH, and autoconnect ensures that the network starts after every reboot. A phone joined the network and reached the Pi at 192.168.1.254.

Wiring and hardware check. The circuit of the previous practice was reused and connected to the header with individual jumpers in BOARD numbering: pins 12, 16, 18, 22, 24, 26 and 32 to inputs IN1–IN7 of the ULN2003, and pins 36, 38, 40 and 37 to inputs A, B, C and D of the 74LS47, with LT and RBI tied to 5 V. Before starting the server, every output was exercised directly from Python (leds(1) to leds(7) and bcd(5)). The first attempt failed: on Debian 13, RPi.GPIO runs on top of lgpio, which could not create its notification pipe without root privileges (*xCreatePipe: Operation not permitted*). Every test was therefore run with sudo, as was the server, which also needs root to bind port 80.

Web server. webserver.py extends BaseHTTPRequestHandler. Its do_GET method returns user_interface.html for the root path and any other existing file with the MIME type detected by python-magic. Its do_POST method decodes a JSON object with the keys action and value, sent asynchronously by the page each time a button is pressed, and dispatches it to the control module through a lookup table:

```
switcher = {'led': leds, 'marquee': marquee,
            'numpad': bcd, 'speed': fijar_velocidad}
func = switcher.get(json_obj['action'], None)
if func:
    func(json_obj['value'])
```

Two changes were made to the base code: the server was bound to port 80, and do_POST now answers every request with 200 OK and Connection: close, since the original handler returned without sending any status line, so the browser never received a valid reply to its request.

Control logic. The code of the previous practice was reorganized in led_manager.py as functions called by the server. Because HTTPServer handles one request at a time, an endless animation executed inside the handler would block it. Each animation therefore runs in a daemon thread, and a threading.Event lets any new command stop the current animation before starting another:

```
def detener_animacion():
    if hilo_actual and hilo_actual.is_alive():
        stop_event.set() # ask the running thread to finish
        hilo_actual.join(0.5)
    stop_event.clear()
    GPIO.output(PINES_LEDS, GPIO.LOW)
```

The five required functions were implemented as follows. Experiment 1 (one LED per button) turns the row off and then lights only the selected LED; experiments 2 and 3 (left and right marquees) traverse the pin list in natural and reverse order; experiment 4 (ping-pong) concatenates a forward pass with a backward pass that skips both ends, so the extreme LEDs are not lit twice; and experiment 5 writes each bit of the digit to one input of the 74LS47:

```
GPIO.output(PINES_LEDS[value - 1], GPIO.HIGH) # Exp. 1
for i in range(len(PINES_LEDS) - 1, -1, -1): ... # Exp. 3
secuencia = list(range(7)) + list(range(5, 0, -1)) # Exp. 4
GPIO.output(36, GPIO.HIGH if (num & 0x01) else GPIO.LOW) # Exp. 5: bit 0 (A)
GPIO.output(37, GPIO.HIGH if (num & 0x08) else GPIO.LOW) # bit 3 (D)
```

All five experiments produced the expected behavior on the circuit when operated from the phone's

browser. A video of the complete test, recorded while the page was operated from the phone, is available at <https://www.youtube.com/watch?v=f7k0TztHO7w>.

Optional exercise 2: variable speed. Every animation waits with `stop_event.wait(tiempo_espera_global)` instead of `sleep()`. This makes the delay interruptible and re-reads the global period on every step, so a new speed received from the interface, either as a speed action or as an optional speed field in any request, takes effect immediately without restarting the animation. In the page, a slider (0.05 to 1.0 s per step, in increments of 0.05 s, 0.2 s by default) sends the selected period when it is released. `fijar_velocidad` converts the value to float and ignores anything that is not a positive number.

Optional exercise 3: setup script. `setup_ap.sh` automates the whole configuration described above. It takes the SSID and the password as optional positional parameters, rejects passwords outside the 8–63 character range allowed by WPA2 [2], disables the standalone `dnsmasq` service, recreates the connection in shared mode with autoconnect enabled, and restarts NetworkManager:

```
SSID="${1:-Raspberry_Ortega}"
PASS="${2:-12345678}"
if [ ${#PASS} -lt 8 ] || [ ${#PASS} -gt 63 ]; then exit 1; fi
systemctl disable --now dnsmasq
nmcli connection add con-name "$CON" type wifi ifname wlan0 ssid "$SSID" ... \
    wifi-sec.psk "$PASS" ipv4.method shared ipv4.address "$IP/24" autoconnect yes
```

Conclusions

The practice applied the infrastructure mode of IEEE 802.11, DHCP address leasing on a private class C network, the semantics of HTTP GET and POST requests, and multithreading to keep a single-threaded server responsive while the hardware runs continuous animations. The main difficulties were verifying the individual jumper connections against the header before powering the circuit, and the root privileges that `lgpio` requires on current Debian releases. The system can be improved in several ways: the interface does not yet reflect the state of the Pi, which could be solved with a GET endpoint queried periodically by the page; a `ThreadingHTTPServer` running as a `systemd` service would remove the need to launch it manually as root; and the default password and plain HTTP should be replaced before any use outside the laboratory.

References

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.
- [2] M. S. Gast, *802.11 Wireless Networks: The Definitive Guide*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA, USA: Pearson Prentice Hall, 2011.
- [4] S. Monk, *Raspberry Pi Cookbook*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [5] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.